1. 1. Dependencies Used And Why

1. sqflite Used to store and manage data locally in SQLite on mobile (Android/iOS). It provides APIs to create tables and perform CRUD operations.
2. path Used to build the correct database file path (for example joining database directory + file name). This avoids hardcoded paths and works across platforms.
3. intl Used for formatting date/time (for example showing post creation date in readable form).
4. sqflite_common_ffi (for desktop support) Used when running Flutter on Windows/Linux/macOS. It initializes SQLite factory for desktop, fixing databaseFactory not initialized.

Why SQLite is necessary for local storage:

It keeps data on the device, so app features work offline.

It is lightweight, fast, and built for structured data (tables/rows).

It supports reliable queries, filtering, sorting, and updates.

It is better than simple key-value storage when data has relationships and needs CRUD.

2. 2. How I Handle Database Exceptions

1. Handle "database not initialized"

Ensure database initialization runs before any query.

On desktop, initialize with FFI before opening DB.

Wrap init code in try-catch and show user-friendly error (SnackBar/message + Retry).

2. Handle insert/update/delete errors

Each CRUD method uses try-catch.

Validate input before operation (title/content/author not empty).

For update/delete, check affected rows: if 0, record may not exist.

Throw meaningful exceptions and display readable feedback in UI.

3. Handle invalid or corrupted data

While reading rows, map each row safely.

If one row is corrupted, skip it instead of crashing whole list.

Use default values or null checks in model conversion (fromMap).

Log error details for debugging, but show simple message to user.

3. SQLite In Flutter

1. Database vs Table

Database: The full file container (example posts.db).

Table: A structure inside the database (example posts table) with columns like id, title, content, author, createdAt.

2. CRUD operations

Create: Insert new post into posts.

Read: Fetch all posts or one post by ID.

Update: Modify existing post using WHERE id = ?.

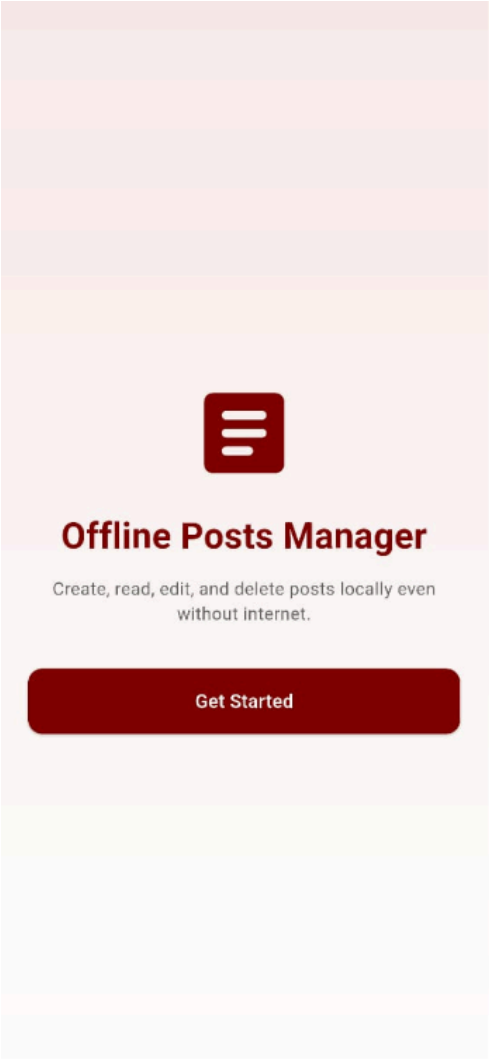
Delete: Remove post using WHERE id = ?.

3. Async interaction in Flutter

- Database calls are asynchronous (Future + await) so UI does not freeze.
- Example flow:
  - UI calls await dbHelper.getAllPosts()
  - Show loading indicator while waiting
  - Update state after result
  - Show error message if exception occurs
- This keeps app responsive and user-friendly.

SQLite is the core of my Offline Posts Manager because it enabled reliable offline CRUD with structured local storage, while Flutter async methods keep the interface smooth and responsive.

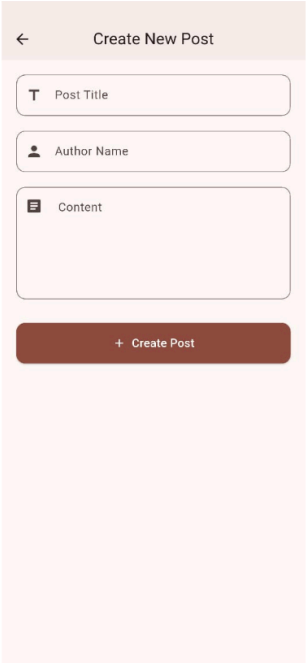
When the app launches, the user first sees a simple Home screen with a Get Started button. After tapping it, a short loading state appears, then the app opens the Posts List screen.

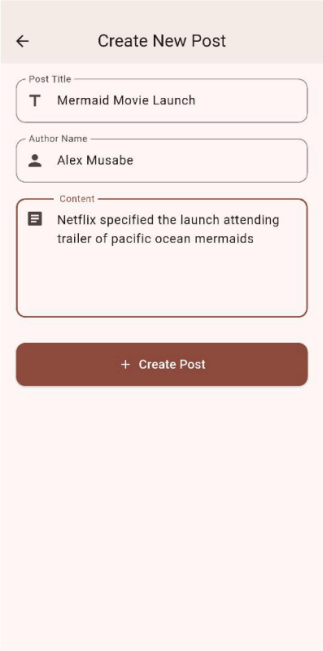


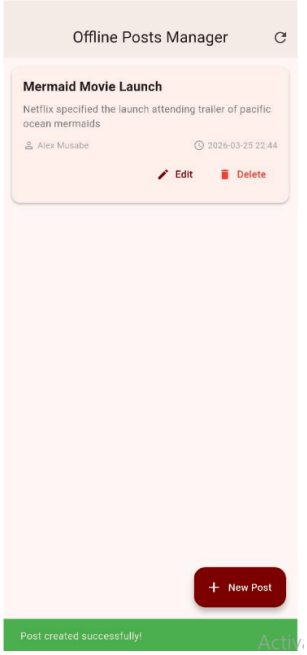
On startup, Flutter initializes the app and prepares the local SQLite database through DatabaseHelper. If the database file does not exist yet, it is created automatically, and the posts table is created. This ensures the app is ready to work even without the internet.

On the Posts List screen, the app loads all posts from the local database asynchronously. While data is loading, a progress indicator is shown. If posts exist, they appear in a list. If no posts exist, an empty-state message is shown with a prompt to create the first post.

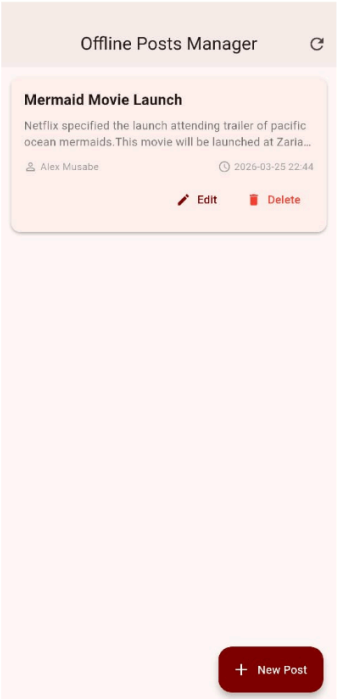
The user can tap New Post to open the Add/Edit screen in create mode. The user enters title, author, and content. The form validates input first, then inserts the post into SQLite. After success, the app returns to the list and refreshes automatically, showing the new post.

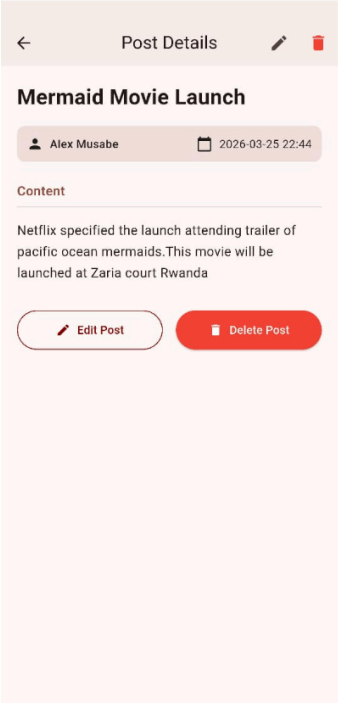






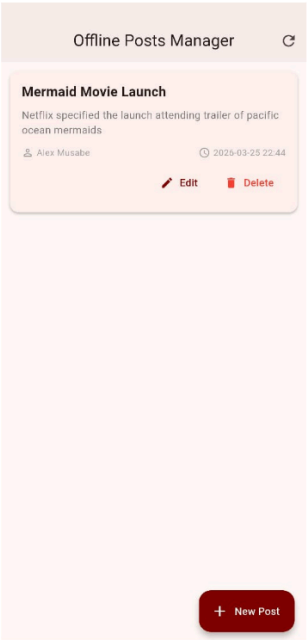
If the user taps a post, the app opens the Post Detail screen. This screen shows full content, author, and date. From here, the user can edit or delete the post.

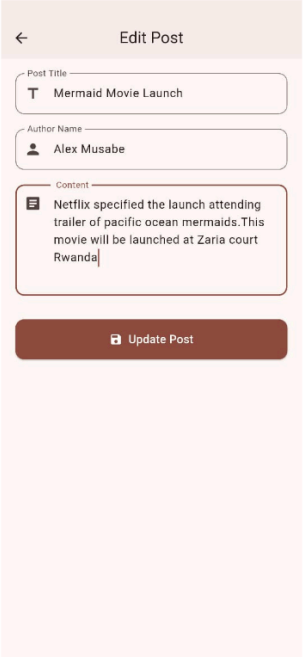


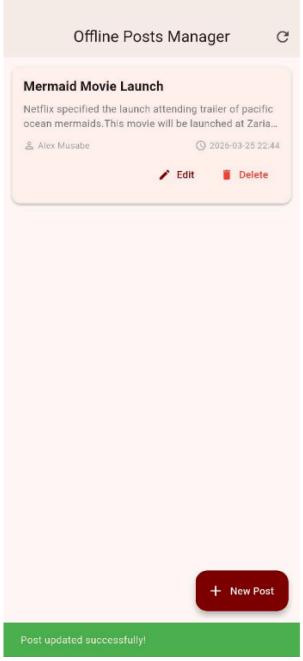


For editing, the app opens the same Add/Edit screen in edit mode with fields prefilled. After the user updates and saves, SQLite runs an update query, then the app refreshes detail/list data to display the latest version.

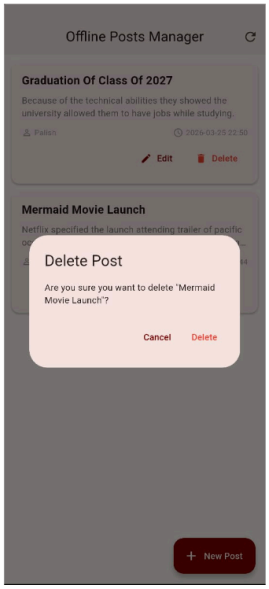
For example this post of mermaid was

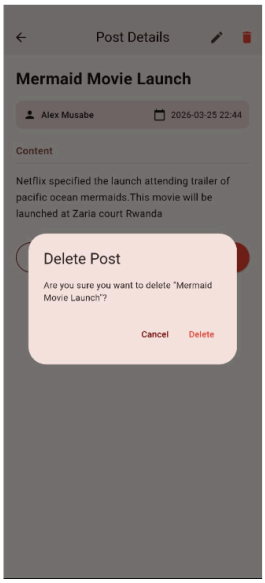


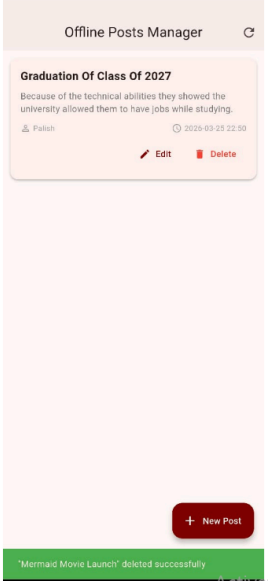


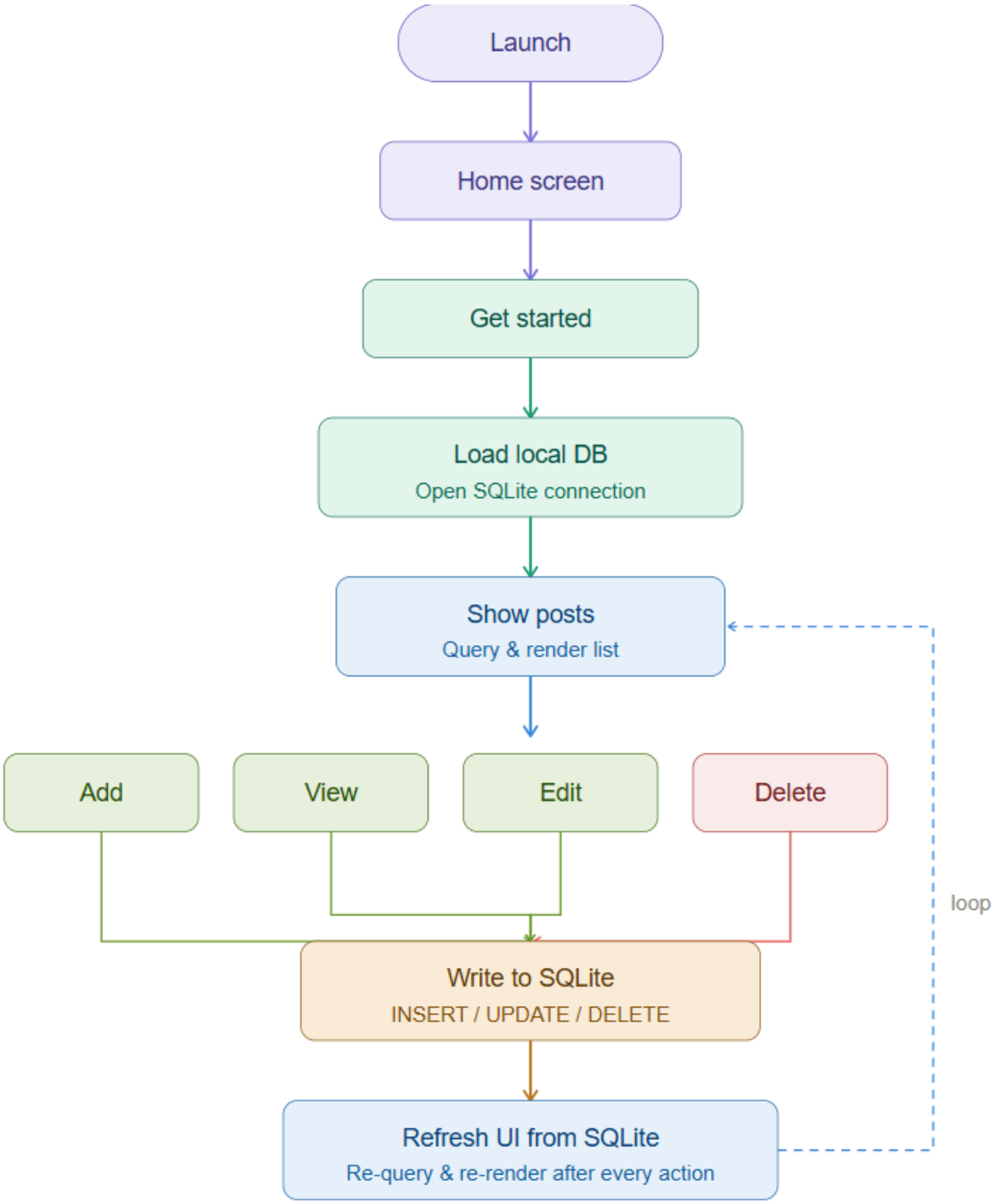


For deleting, the app asks for confirmation first. If confirmed, SQLite removes the record by ID, then the app returns to the list and refreshes so the deleted post no longer appears.(different place you can delete apost)







All database actions (create, read, update, delete) are done asynchronously using Future and await. This keeps the UI responsive and prevents freezing. Errors are caught with try-catch blocks and shown as user-friendly messages (for example SnackBars or retry prompts), so the app can recover safely from failures.

The flow is: